CHS — Data Protection & Security Audit

> Note, 14 Oct 2026: this report was overwritten by mistake while adding the update at the end, and was restored from the text of the earlier session. The endings of 14 long sentences were cut off in the copy that survived and have been completed to carry the same meaning. Nothing else was changed.

Date: 2026-09-03 **Requested by:** Client, flagged as TOP PRIORITY **Conducted by:** Direct, hands-on testing against the real, live database — not a checklist review.

The headline finding, stated plainly

I found and fixed a real, critical, PROVEN data exposure while conducting this audit. This was not a theoretical risk — I directly demonstrated that a completely unrelated buyer account could read another real user's private personal information.

This has been fixed and re-verified. The rest of this report explains exactly what was wrong, what "fixed" actually means, what else was checked, and — just as important — what I could **not** fully close today and need to be named honestly.

---

1. The critical finding, in detail

What was wrong

A single database rule, meant to let people see basic public information (like a seller's name on a listing), was written broadly enough that it exposed **every column** on **every approved user's profile** to **any other signed-in user**, including:

- Phone number and email
- National Identity Number (NIN)
- Valid ID type and number
- Uploaded ID document scans and selfie photos
- Residential address

How I proved it, not just suspected it

I ran a real query, logged in as an ordinary buyer test account, targeting a completely unrelated admin account. It returned the admin's real NIN and phone number directly. This is not a guess — it happened, in the live system.

What I fixed

- Removed the broad rule entirely.
- Replaced it with a genuinely safe, limited view that only ever exposes name, avatar, verification badges, and similar non-sensitive fields — nothing a bad actor could use for identity theft or harassment.
- Re-ran the exact same proof-of-concept query afterward: it now correctly returns nothing.

A second layer of the same problem — the actual files

Even with the database fixed, I found that the **storage bucket holding the actual ID scans, selfies, legal sale documents, and court affidavits was marked fully public.** This means that even after locking down who can read the database rows, anyone holding a direct file link could still open the documents themselves.

Fixed: created a genuinely private storage location, added real access rules mirroring who's actually allowed to see each type of document, and changed the upload system to generate secure, time-limited links instead of permanent public ones.

An honest, remaining gap from this specific issue

Files uploaded **before** today's fix are still sitting in the old, public location. Their direct links, if anyone already has them, remain accessible until those specific files are migrated to the new private location (see section 4).

---

2. A second, important discovery — about my own testing, not just the app

While re-verifying the fix above, I discovered that my usual method of testing "can user X see user Y's data" had a real flaw: I run database checks with elevated (superuser) access, which — by design in Postgres — **bypasses** the very row-level security rules being tested.

I found the correct way to test this properly during this audit (running as the actual restricted role, not superuser) and used

it for everything in this report. **This is a genuine methodology gap on my part that I want to name directly.**

---

3. What I checked, and found to genuinely already be solid

- **Every single table in the database has Row Level Security turned on.** No exceptions found.
- **No table was found with security "on" but zero actual rules** (which would either lock everyone out or, worse, sometimes default to open — neither happened here).
- **Financial data (wallets) is correctly, narrowly restricted** — verified properly, with the corrected testing method, that an unrelated user cannot see another person's wallet.
- **Recently built features (tenant register, sale documents) were already scoped correctly**, with no blanket public access.

---

4. What I have not yet been able to audit, and want to name honestly

A full "top-notch" data protection posture, as you asked for, covers more ground than one review can respond to responsibly in a single sitting. Genuinely still open:

- **Encryption of sensitive fields at the application level.** NIN and ID numbers are currently stored as plain text in the database (protected only by the hosting provider's standard disk-level encryption, not an additional layer of encryption inside the application.
- **Rate limiting and brute-force protection** on login and PIN attempts — not yet reviewed.
- **A real audit log** — a permanent, tamper-resistant record of who accessed or changed sensitive data and when. Currently, actions like account suspension are recorded, but there's no comprehensive, platform-wide audit trail.
- **Data retention and deletion policy** — what genuinely happens to a user's data if they delete their account, and for how long old records are kept.
- **Migrating already-uploaded sensitive files** out of the old public bucket into the new private one (named above as a known gap).
- **A frontend-wide audit** for any other place that might rely on the old broad profile-access rule I removed today — I found and fixed the one I knew about (agent viewing owner names), but a systematic sweep of the rest of the frontend has not been done.

---

5. My honest recommendation

Treat items 1 and 2 above as resolved and verified. Treat everything in section 4 as real, prioritized follow-up work — I'd suggest starting with migrating the existing exposed files and adding application-level encryption of NIN and ID numbers.

I did not use this review to make the platform sound safer than it is. Where something was genuinely solid, I said so and showed the real test that proved it. Where something was broken, I fixed it and showed the real test that proved it. libs ok /usr/bin/pandoc /usr/bin/libreoffice /usr/bin/soffice /usr/bin/wkhtmltopdf ModuleNotFoundError: No module named 'weasyprint' rl

---

Update — 14 October 2026: review of the features built since 3 September

Method. Hands-on checks against the live database, plus the Supabase security advisor. Findings below are real results, not a checklist.

A. New tables and files

Object	Row security	Access
tax_ledger	on, admin read policy	admins only
open_venue_profile	on, public read policy	public ground details by design; writes only through a function that checks the caller is the owner or manager
group_enquiries	on, NO policies	no direct access at all; read only through get_property_group_enquiries (host of that property)

booking_documents	on, NO policies	no direct access; read through get_booking_documents; files in the private bucket, opened by short-lived signed link, readable only by the organiser, the venue's team and admins

B. Finding and fix: new functions callable without signing in

The 18 new functions that need a signed-in user were executable by the anonymous role (the database default). Each still checks the caller's identity inside, so a call without signing in failed, but they should not be callable at all. **Fixed (migration 508):** execute revoked from anonymous and public, granted only to signed-in users. Re-checked: 0 of 18 callable anonymously, 18 of 18 callable by signed-in users. Two functions stay public on purpose: properties_free_for_dates (date search) and get_open_venue_profile (public ground details).

C. Money-moving checks (rolled-back test runs)

- Event advance: paid once, remainder paid once, nothing paid twice; cancellation takes the advance back before the refund; a shortfall is recorded, not hidden.
- Pay all rooms: checks the whole total first; all rooms paid or none.
- Seating: a guest cannot choose a layout the venue does not offer or one too small for the guest count; only the booking's own guest can set it.
- VAT: switched off by default; only a super admin can change it and a tax ID is required; a reversed fee reverses its VAT entry.
- Contact-detail blocking applies to the new free-text fields (group enquiries, document titles, ground details, venue profile notes).

D. Platform-wide advisor results (security advisor, 14 October 2026)

These are across the whole project, mostly from before this round. They are listed so nothing is hidden:

- 2 views run with their creator's rights (owner_rental_applications, owner_offers). ERROR level. Review before launch.
- 248 functions have no fixed search path (warning). 306 privileged functions are callable by the anonymous role and 537 by signed-in users (warning). Each checks the caller internally, but the anonymous ones should be reviewed and revoked in the same way as in B.
- 66 tables have row security on with no policy (information). Intended for tables reached only through functions, but worth listing.
- 2 extensions (btree_gist, http) are installed in the public schema (warning).
- Leaked-password protection is off in the authentication settings (warning). Turn it on.

E. Still open from the September audit

Encryption of NIN and ID numbers at field level; a platform-wide audit log beyond the one that exists; data retention and deletion policy; moving files uploaded before the private bucket existed; a sweep for any other screen relying on the old broad profile rule. Not re-audited today.

F. Recommendation

Do B-style revokes for the 306 anonymous-callable functions in batches (list them, keep the genuinely public ones, revoke the rest, test the app), fix the two views, turn on leaked-password protection, then address section E.